

Programmierkurs



Einführung in C# Tag 4

Inhaltsübersicht Vorkurs



- Tag 1: Zustände, Variablen, Datentypen, Konvertierungen, Arithmetik, Visual Studio Demo
- Tag 2: Kommentare, Boolesche Ausdrücke, If-Abfragen, Switch-Case
- Tag 3: Arrays, (Do-)While-Schleife, For-Schleifen, Weiterführung Debugging
- Tag 4: (statische) Methoden, Klassenvariablen

Ablauf



- 09:30 Lösungen des Vortages
- ab 10:00 Vorlesung
- 60 min Mittagspause
- gegen 12:30 / 13:00 Vorstellung Aufgabenstellung + Übungen
- ca 15 Uhr: Vorstellung Lösungen Tag 4
- danach: Come Together

Beispiel ohne Methoden



```
public class Main {  
    public static void main () {  
        for (int i = 1; i <= 10; i++) {  
            Console.WriteLine(i);  
        }  
        // ...  
        for (int i = 1; i <= 10; i++) {  
            Console.WriteLine(i);  
        }  
    }  
}
```

Warum ist dieser Code problematisch?



Probleme:

- Zeitaufwändig
- (zu) viel Code
- Unübersichtlich
- Änderungen kosten noch mehr Zeit
- Code oft nicht wiederverwendbar
- Arbeitsteilung kaum möglich

Warum ist dieser Code problematisch?



Probleme:

- Zeitaufwändig
- (zu) viel Code
- Unübersichtlich
- Änderungen kosten noch mehr Zeit
- Code oft nicht wiederverwendbar
- Arbeitsteilung kaum möglich

Lösungen:

- Ähnlichen Code auslagern
- Wiederverwendbaren Code schreiben
- **Methoden!**

Beispiel mit Methoden



```
public class Main {  
    public static void main () {  
  
        zaehlBisZehn ();  
        // ...  
        zaehlBisZehn ();  
    }  
  
    public static void zaehlBisZehn () {  
        for (int i = 1; i <= 10; i++) {  
            Console.WriteLine(i);  
        }  
    }  
}
```

Methodenaufbau



```
public static Rückgabetyp Name(Parameter){  
    // Methodenrumpf  
    return ... ;  
}
```

Beispiel



```
public static void zaehlBisZehn () {  
    // ...  
}
```

Anfang: `public static`

Rückgabotyp: `void`

Methodenname: `zaehlBisZehn`

Parameter: keine



Aufruf einer Methode

```
public static void main () {  
    zaehlBisZehn ();  
    int x = gibMir42 ();  
    int y = verdoppelWert (x);  
}
```

- Methodenname
- Parameter
- ;

Beispiel ohne Methoden



```
public class Main () {  
    public static void main () {  
        for (int i = 1; i <= 9; i++) {  
            Console.WriteLine(i);  
        }  
        // ...  
        for (int i = 1; i <= 10; i++) {  
            Console.WriteLine(i);  
        }  
    }  
}
```

Beispiel mit Methoden



```
public class Main () {  
    public static void main () {  
        zaehleBis (9);  
        // ...  
        zaehleBis (10);  
    }  
  
    public static void zaehleBis (int zahl) {  
        for (int i = 1; i <= zahl; i++) {  
            Console.WriteLine(i);  
        }  
    }  
}
```

Parameter

```
public static void zaehleBis (int zahl) {  
    // ...  
}
```

- In die runden Klammern kommen die Parameter
- Parameter werden mit Komma getrennt:
(int a, boolean b, double c)
- Ein Parameter besteht aus Datentyp und Bezeichner

Was passiert?



➤ `zaehleBis(9);`

```
public static void zaehleBis (int zahl) {  
    // " zahl " wird der Wert 9 zugewiesen  
}
```

➤ `zaehleBis(10);`

```
public static void zaehleBis (int zahl) {  
    // " zahl " wird der Wert 10 zugewiesen  
}
```

Beispiel mit 2 Parametern



```
public class Main {  
    public static void main () {  
        zaehleVonBis (1, 9);  
        // ...  
        zaehleVonBis (5, 10);  
    }  
  
    public static void zaehleVonBis (int von, int bis) {  
        for (int i = von; i <= bis; i++) {  
            Console.WriteLine(i);  
        }  
    }  
}
```

Beispiel mit Rückgabewert



```
public class Main {  
    public static void main () {  
        int x = gibMir42 ();  
    }  
  
    public static int gibMir42 () {  
        return 42;  
    }  
}
```



Rückgabewert

```
public static int gibMir42 () {  
    return 42;  
}
```

- Möchte man einen Wert zurück geben, so wird der entsprechende Datentyp angegeben (z.B. `int`, `double`, `string`)
- Wenn es keinen Rückgabewert gibt wird das Schlüsselwort `void` als Datentyp angegeben
- Wenn es einen Rückgabewert gibt wird dieser in der Methode mit dem Befehl `return` zurückgegeben

Was passiert?



```
int x = gibMir42 ();
```

- Rechte Seite von "=" wird zuerst ausgewertet
 - gibMir42();

```
public static int gibMir42 () {  
    return 40 + 2;  
}
```

- return 40 + 2
 - return 42
 - 42 wird zurückgegeben
- x wird der Wert 42 zugewiesen

Lesbarkeit durch Methoden



- Berechnung der Fakultät: $n! = 1 \times 2 \times \dots \times n$

```
public static long fak (int n){  
    long faku = 1;  
    // Iterative Berechnung  
    for(int i = 1; i <= n; i++) {  
        faku *= i;  
    }  
    return faku ;  
}
```

Binomialkoeffizient berechnen

$$\binom{n}{k} = \frac{n!}{(n-k)! \times k!}$$

- Für die Berechnung braucht man 3 Mal eine Fakultät
- Somit 3 Schleifen in der Berechnung
- Code unlesbar

Binomialkoeffizient berechnen

$$\binom{n}{k} = \frac{n!}{(n-k)! \times k!}$$

- Für die Berechnung braucht man 3 Mal eine Fakultät
- Somit 3 Schleifen in der Berechnung
- Code unlesbar

Lösung:

- Methode `long fak (int n)` benutzen

Binomialkoeffizient berechnen

$$\binom{n}{k} = \frac{n!}{(n-k)! \times k!}$$

```
public static double binomial (int n, int k) {  
    double bin = fak(n) / (fak(n - k) * fak(k));  
    return bin;  
}
```

Beispiel ohne Klassenvariablen



```
public class Vorkurs {  
    public static void main () {  
        int x = zaehleVonBis (1, 9);  
        Console.WriteLine( wurdeAusgegeben );  
        // Das funktioniert so leider nicht  
    }  
  
    public static int zaehleVonBis (int von , int bis) {  
        for (int i = von; i <= bis; i++) {  
            Console.WriteLine(i);  
        }  
  
        bool wurdeAusgegeben = bis >= von;  
        return bis - von + 1;  
    }  
}
```

Warum?



Probleme:

- Methoden können nur einen Wert zurückgeben
- Eine Methode kann nicht auf Daten aus anderen Methoden zugreifen

Warum?



Probleme:

- Methoden können nur einen Wert zurückgeben
- Eine Methode kann nicht auf Daten aus anderen Methoden zugreifen

Lösung

- **Klassenvariablen**
 - mit Bedacht verwenden!

Beispiel mit Klassenvariablen



```
public class Vorkurs {  
    public static bool wurdeAusgegeben;  
  
    public static void main () {  
        int x = zaehleVonBis (1, 9);  
        Console.WriteLine(wurdeAusgegeben);  
    }  
  
    public static int zaehleVonBis (int von , int bis) {  
        for (int i = von; i <= bis; i++) {  
            Console.WriteLine(i);  
        }  
        wurdeAusgegeben = bis >= von ;  
        return bis - von + 1;  
    }  
}
```

Deklaration von Klassenvariablen

- Deklaration direkt nach Klassendeklaration
- **public static** `Datentyp Variablenname`;
- Sichtbar in der ganzen Klasse
- Ohne manuelle Zuweisung wird der Defaultwert zugewiesen
- Beispiel:

```
public static boolean wurdeAusgegeben;
```

Quellen



Original von Samuel Zeitvogel

Überarbeitet 2012 von Daniel Hoff

Überarbeitet 2013 von Tristan Wagner

Überarbeitet 2015 von Tobias Kerst

Überarbeitet 2016 von Christian Wernet

Überarbeitet 2018 von Jan Oliver Zerulla

Überarbeitet 2018 von Johannes Beierle

Überarbeitet 2020 von Jonas Westenhoff

Überarbeitet 2021 von Tobias Hnyk